Métodos de Optimización

Clase 5: Optimización no lineal con restricciones

[Nombre del profesor]

[Nombre de la universidad / programa]

Contenido

Motivación y panorama

Condiciones KKT

Programación cuadrática (QP)

Cutting plane (planos de corte)

Programación cuadrática secuencial (SQP)

Métodos de penalización: exterior e interior

Síntesis y cierre del curso

Motivación y panorama

De dónde venimos

- **Clases 1–3**: conceptos (gradiente, Hessiana, convexidad), búsqueda 1D y metaheurísticas.
- **Clase 4**: métodos de dirección de búsqueda *sin restricciones*: gradiente, Newton, BFGS, trust region, Levenberg–Marquardt, SGD.
- **Hoy (Clase 5)**: ¿qué cambia cuando x **no puede tomar cualquier valor**?

Idea central de hoy

Con restricciones, el mínimo ya **no es un punto donde** $\nabla f = 0$. Todo lo que sigue consiste en: **(1)** entender qué condiciones cumple el mínimo (KKT) y **(2)** convertir el problema en otros que *ya sabemos resolver* (sistemas lineales, QPs, LPs o problemas sin restricciones).

¿Por qué aparecen restricciones?

- **Límites físicos**: concentraciones ≥ 0, tasas de decaimiento $b, d > 0$ (¡como en el miniproyecto de ajuste!).
- **Diseño**: minimizar costo/peso con límites de esfuerzo, temperatura máxima, caída de presión.
- **Balances** (igualdades): masa, energía, ecuaciones de estado.
- **Recursos**: presupuesto, potencia, tiempo.

Ejemplo en una línea
Elegir espesor de aislante e para minimizar el costo $c(e)$ *sujeto a* que la temperatura superficial no supere $60\,^{\circ}\mathrm{C}$: $T_s(e) \leq 60$.

Con restricciones, el óptimo casi siempre queda **justo en el borde** de lo permitido: la restricción "se activa".

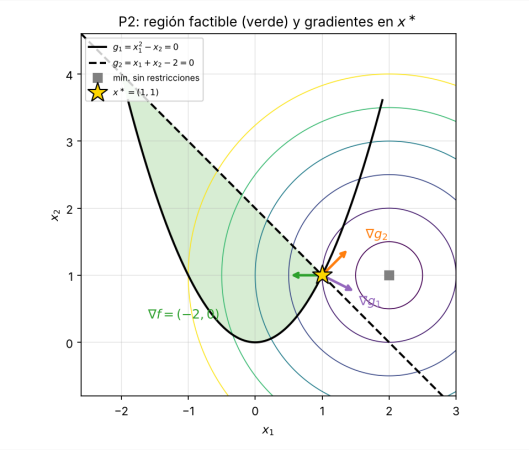
El problema general (NLP)

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{sujeto a} \quad \underbrace{h_i(x) = 0,\ i = 1, \ldots, m}_{\text{igualdades}}, \qquad \underbrace{g_j(x) \leq 0,\ j = 1, \ldots, p}_{\text{desigualdades}}$$

- **Región factible**: $\Omega = \{x : \ h(x) = 0,\ g(x) \leq 0\}$. Un punto es **factible** si está en Ω.
- Convención de hoy: las desigualdades se escriben siempre como $g_j(x) \leq 0$ (si viene $c(x) \geq 0$, se usa $g = -c$).
- Una desigualdad es **activa** en x si $g_j(x) = 0$ (el punto toca el borde) e **inactiva** si $g_j(x) < 0$ (hay holgura).

Notación del curso
$\lambda \in \mathbb{R}^m$: multiplicadores de las **igualdades**. $\mu \in \mathbb{R}^p$: multiplicadores de las **desigualdades**. Lagrangiano: $L(x, \lambda, \mu) = f(x) + \lambda^\top h(x) + \mu^\top g(x)$.

Con restricciones, $\nabla f(x^*) \neq 0$



Ejemplo P2

$$\text{mín } (x_1 - 2)^2 + (x_2 - 1)^2$$

$$\text{s.a.} \quad x_1^2 - x_2 \leq 0, \quad x_1 + x_2 - 2 \leq 0$$

- Sin restricciones el mínimo sería $(2, 1)$: está **fuera** de la región verde.
- El mínimo con restricciones es $x^* = (1, 1)$: está en el **borde**, en la intersección de las dos restricciones.
- Allí $\nabla f = (-2, 0) \neq 0$: la función *quisiera* seguir bajando hacia la derecha, pero las restricciones se lo impiden.

Mapa de la clase: ¿qué hace cada método con las restricciones?

Método	Reemplaza el problema por...	Se resuelve con...
KKT	condiciones que cumple el óptimo	ecuaciones (a mano o numérico)
Programación cuadrática (QP)	objetivo cuadrático + restricciones lineales	sistema lineal / conjunto activo
Cutting plane	problema **lineal** (LP) por linearización	LP repetidos
SQP	un **QP** en cada iteración (Newton sobre KKT)	QPs repetidos
Penalización exterior / interior	secuencia de problemas **sin** restricciones	BFGS / Newton (Clase 4)

Todos se apoyan en lo mismo: **aproximar localmente** (linealizar, cuadratizar o penalizar) para volver a problemas fáciles.

Problemas de prueba de la clase

P1 — una igualdad
mín $x_1 + x_2$ s.a. $x_1^2 + x_2^2 = 2$
Solución: $x^* = (-1, -1)$, $f^* = -2$, $\lambda^* = \frac{1}{2}$.

P2 — dos desigualdades
mín $(x_1 - 2)^2 + (x_2 - 1)^2$
s.a. $x_1^2 \le x_2$, $x_1 + x_2 \le 2$
Solución: $x^* = (1, 1)$, $f^* = 1$, $\mu^* = (\frac{2}{3}, \frac{2}{3})$.

P3 — una desigualdad curva
mín $(x_1 - 2)^2 + (x_2 - 1)^2$ s.a. $x_1^2 + x_2^2 \le 1$
Solución: $x^* = \frac{1}{\sqrt{5}}(2, 1)$,
$f^* = (\sqrt{5} - 1)^2 \approx 1{,}528$, $\mu^* = \sqrt{5} - 1$.

P4 — no convexo
Rosenbrock $(1 - x_1)^2 + 100(x_2 - x_1^2)^2$ dentro del disco $x_1^2 + x_2^2 \le 1$.
Solución: $x^* \approx (0{,}7864, 0{,}6177)$, $f^* \approx 0{,}0457$.

En cada código: parte **manual** + comparación con `scipy.optimize` (`SLSQP`, `trust-constr`).

Condiciones KKT

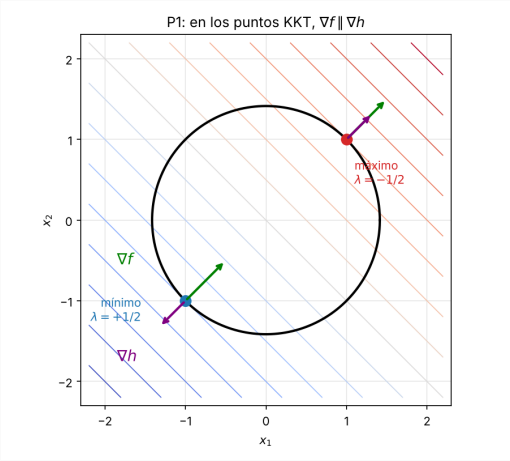
Recordatorio: sin restricciones

- En un mínimo local **sin restricciones**: $\nabla f(x^*) = 0$ (si no, podríamos bajar moviéndonos en la dirección $-\nabla f$).
- Pregunta de hoy: ¿qué **ecuación equivalente** cumple el mínimo cuando hay restricciones?

Estrategia para entenderlo
Vamos por pasos: **(1)** una restricción de igualdad $\rightarrow$ multiplicador de Lagrange; **(2)** una desigualdad $\rightarrow$ el multiplicador debe ser ≥ 0; **(3)** varias restricciones $\rightarrow$ condiciones KKT completas.

Idea base de todo: **en un mínimo no se puede mejorar f sin romper alguna restricción.**

Una igualdad: caminar sobre una curva



P1: mín $x_1 + x_2$ sobre la circunferencia $x_1^2 + x_2^2 = 2$.

- Solo podemos movernos **a lo largo de la curva**.
- Si ∇f tiene componente a lo largo de la curva, podemos deslizarnos y bajar $f \Rightarrow$ no es mínimo.
- En el mínimo, ∇f no tiene componente tangente: es **perpendicular a la curva**.
- Pero ∇h *también* es perpendicular a la curva. Entonces:

$$\nabla f(x^*) \parallel \nabla h(x^*)$$

El multiplicador de Lagrange

$\nabla f \parallel \nabla h$ significa que existe un número λ tal que

$$\nabla f(x^*) + \lambda \nabla h(x^*) = 0, \qquad h(x^*) = 0.$$

El Lagrangiano junta todo en una sola función

$$L(x, \lambda) = f(x) + \lambda\, h(x) \qquad \Longrightarrow \qquad \nabla_x L = 0, \quad \nabla_\lambda L = h = 0$$

- Son $n + m$ ecuaciones para $n + m$ incógnitas (x y λ): un sistema (no lineal) **sin restricciones**.
- Con varias igualdades: $\nabla f + \sum_i \lambda_i \nabla h_i = 0$ (∇f es combinación lineal de los ∇h_i).
- λ **puede tener cualquier signo** (en igualdades).

Resolviendo P1 a mano

$$L = x_1 + x_2 + \lambda\,(x_1^2 + x_2^2 - 2)$$

$$\frac{\partial L}{\partial x_1} = 1 + 2\lambda x_1 = 0, \qquad \frac{\partial L}{\partial x_2} = 1 + 2\lambda x_2 = 0, \qquad x_1^2 + x_2^2 = 2$$

De las dos primeras: $x_1 = x_2 = -\dfrac{1}{2\lambda}$. En la tercera: $\dfrac{2}{4\lambda^2} = 2 \Rightarrow \lambda^2 = \dfrac{1}{4}$.

$\lambda = +\frac{1}{2}$
$x = (-1, -1)$, $f = -2$
mínimo

$\lambda = -\frac{1}{2}$
$x = (1, 1)$, $f = +2$
máximo

Las condiciones son **necesarias pero no suficientes**: el sistema encuentra *todos* los puntos donde $\nabla f \parallel \nabla h$ (mínimos, máximos y puntos silla). Hay que distinguir después (curvatura, convexidad o comparar valores).

¿Qué significa λ? El precio sombra

Relajemos la restricción: $h(x) = \varepsilon$ en lugar de $h(x) = 0$. Sea $f^*(\varepsilon)$ el nuevo valor óptimo. Con $L = f + \lambda h$:

$$\boxed{\left.\frac{df^*}{d\varepsilon}\right|_{\varepsilon=0} = -\lambda^*}$$

Comprobación con P1

Con $h = \varepsilon$: $x_1^2 + x_2^2 = 2 + \varepsilon$, y el mínimo es $f^*(\varepsilon) = -\sqrt{4 + 2\varepsilon}$.

$$\left.\frac{df^*}{d\varepsilon}\right|_0 = -\frac{1}{\sqrt{4}} = -\frac{1}{2} = -\lambda^* \ \checkmark$$

- λ mide **cuánto cambia el óptimo si se afloja la restricción**: es el "precio" de la restricción.
- Ingeniería: λ grande $\Rightarrow$ la restricción que más "duele": conviene relajarla.

Desigualdades: activa o inactiva

Una restricción $g(x) \leq 0$ solo puede estar en uno de dos estados en el óptimo x^*:

Inactiva: $g(x^*) < 0$
El punto está **dentro**, con holgura. La restricción "no se entera": localmente el problema es como uno sin restricciones.

$\Rightarrow$ **no aporta fuerza**: $\mu = 0$.

Activa: $g(x^*) = 0$
El punto está **en el borde**. La restricción se comporta como una igualdad, pero con una diferencia importante...

$\Rightarrow$ puede aportar fuerza $\mu \geq 0$.

Analogía: una bolita y una valla
f es el terreno, la bolita rueda hacia abajo y una valla ($g \leq 0$) le impide pasar. Si la bolita queda apoyada en la valla, **la valla la empuja hacia adentro**. Una valla **solo puede empujar, no jalar**: por eso $\mu \geq 0$.

Por qué $\mu \geq 0$: solo se puede empujar hacia adentro

En el borde, una dirección d es **factible** si no nos saca de la región: $\nabla g^\top d \leq 0$.
En un mínimo **no existe** una dirección factible que baje f:

$$\text{no hay } d \text{ con} \quad \nabla f^\top d < 0 \quad \text{y} \quad \nabla g_j^\top d \leq 0 \quad (j \text{ activas}).$$

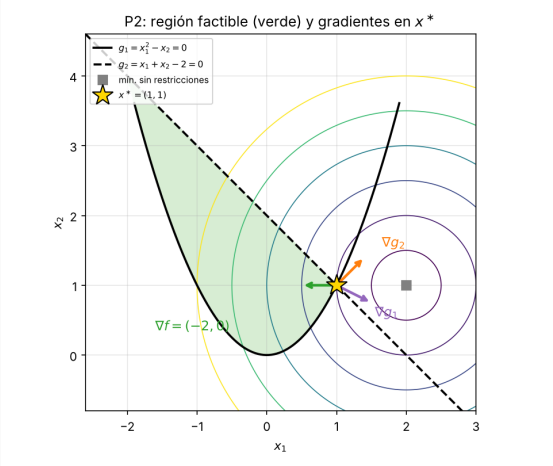
Eso equivale (lema de Farkas) a que $-\nabla f$ sea combinación **con coeficientes no negativos** de los ∇g_j activos:

$$-\nabla f(x^*) = \sum_{j \text{ activas}} \mu_j \nabla g_j(x^*), \qquad \mu_j \geq 0.$$

Interpretación
$-\nabla f$ es hacia donde **quiere ir** la función. Cada ∇g_j apunta hacia **afuera**. La fuerza total de las vallas ($\mu_j \nabla g_j$) compensa exactamente el deseo de ir hacia afuera.

Si algún $\mu_j < 0$
La restricción j estaría "jalando" hacia el borde: eso significa que **soltándola** podríamos mejorar f. Esta es la regla que usan los métodos de conjunto activo.

Geometría en P2: $-\nabla f$ dentro del cono de los ∇g



En $x^* = (1, 1)$ las dos restricciones están activas:

$$\nabla f = (-2, 0), \quad \nabla g_1 = (2, -1), \quad \nabla g_2 = (1, 1)$$

Buscamos $\mu_1, \mu_2 \geq 0$ tales que $-\nabla f = \mu_1 \nabla g_1 + \mu_2 \nabla g_2$:

$$\begin{cases} 2\mu_1 + \ \mu_2 = 2 \\ -\mu_1 + \ \mu_2 = 0 \end{cases} \Rightarrow \mu_1 = \mu_2 = \tfrac{2}{3} \ \geq 0 \ \checkmark$$

$-\nabla f = (2, 0)$ cae **dentro del cono** generado por ∇g_1 y ∇g_2: no hay forma de bajar f sin salirse de la región.

Las condiciones KKT (Karush–Kuhn–Tucker)

Para mín $f(x)$ s.a. $h(x) = 0,\ g(x) \leq 0$, si x^* es mínimo local (y las restricciones son "regulares") existen λ^*, μ^* tales que:

1. **Estacionariedad:** $\nabla f(x^*) + \sum_i \lambda_i^* \nabla h_i(x^*) + \sum_j \mu_j^* \nabla g_j(x^*) = 0$
2. **Factibilidad primal:** $h(x^*) = 0, \qquad g(x^*) \leq 0$
3. **Factibilidad dual:** $\mu_j^* \geq 0 \quad \forall j$
4. **Complementariedad:** $\mu_j^*\, g_j(x^*) = 0 \quad \forall j$

- (1) es $\nabla_x L = 0$. (2) dice que x^* es un punto permitido.
- (3) es "las vallas solo empujan". (4) dice: **o la restricción está inactiva ($g < 0$) y no actúa ($\mu = 0$), o está activa ($g = 0$) y puede actuar ($\mu \geq 0$)**.

Las cuatro condiciones en lenguaje sencillo

Condición	Qué dice
Estacionariedad	"La fuerza que quiere llevarme a un mejor f ($-\nabla f$) está equilibrada por las fuerzas de las restricciones"
Factibilidad primal	"Estoy en un punto permitido"
Factibilidad dual	"Las vallas solo empujan hacia adentro (nunca jalan)"
Complementariedad	"Una valla que no estoy tocando no ejerce fuerza"

Cómo se usarán hoy

(a) verificar si un candidato es óptimo; **(b)** resolver problemas pequeños por casos; **(c)** *todos los algoritmos de hoy son formas de resolver las KKT.*

Uso 1: verificar un candidato en P2

Candidato $x = (1, 1)$, $\mu = (\frac{2}{3}, \frac{2}{3})$:

Condición	Cálculo	¿Cumple?
1. Estacionariedad	$(-2, 0) + \frac{2}{3}(2, -1) + \frac{2}{3}(1, 1) = (0, 0)$	sí
2. Factibilidad primal	$g_1 = 1 - 1 = 0 \leq 0, \quad g_2 = 1 + 1 - 2 = 0 \leq 0$	sí
3. Factibilidad dual	$\mu = (\frac{2}{3}, \frac{2}{3}) \geq 0$	sí
4. Complementariedad	$\mu_1 g_1 = 0, \ \mu_2 g_2 = 0$	sí

Conclusión
$(1, 1)$ cumple KKT. Como P2 es **convexo** (f convexa, g_1, g_2 convexas), KKT son **suficientes**: $(1, 1)$ es el **mínimo global**.

Uso 2: resolver por casos (¿qué restricciones están activas?)

Con p desigualdades hay 2^p posibilidades de conjunto activo. Para P2: $2^2 = 4$.

Activas	Solución de estacionariedad	Verificación	¿KKT?
ninguna	$x = (2, 1)$	$g = (3, 1)$: **infactible**	no
solo g_1	$x = (1{,}165, 1{,}358)$, $\mu_1 = 0{,}72$	$g_2 = 0{,}52 > 0$: **infactible**	no
solo g_2	$x = (1{,}5, 0{,}5)$, $\mu_2 = 1$	$g_1 = 1{,}75 > 0$: **infactible**	no
g_1, g_2	$x = (1, 1)$, $\mu = (\frac{2}{3}, \frac{2}{3})$	factible, $\mu \geq 0$	**sí**
g_1, g_2	$x = (-2, 4)$, $\mu = (-4{,}7, -10{,}7)$	$\mu < 0$	no

- Para cada conjunto activo se resuelve un sistema de *igualdades*; luego se descartan los casos con infactibilidad primal o $\mu < 0$.
- Funciona para problemas chicos, pero 2^p **crece exponencialmente**: los algoritmos de hoy evitan esta enumeración.

Precio sombra en P2 (comprobación numérica)

Relajamos g_2: $x_1 + x_2 \leq 2 + \varepsilon$ y resolvemos de nuevo (con `SLSQP`):

ε	$f^*(\varepsilon)$
$-0{,}10$	1.07296
$-0{,}01$	1.00673
0	1.00000
$+0{,}01$	0.99340
$+0{,}10$	0.93964

Pendiente numérica en $\varepsilon = 0$:

$$\frac{f^*(0{,}01) - f^*(0)}{0{,}01} = -0{,}660, \quad \frac{f^*(0) - f^*(-0{,}01)}{0{,}01} \approx -0{,}673.$$

Coincide con $-\mu_2^* = -\frac{2}{3} \approx -0{,}667$

Cada unidad que se afloje $x_1 + x_2 \leq 2$ reduce el óptimo en $\approx \frac{2}{3}$. Para las desigualdades también: $df^*/d\varepsilon = -\mu^*$.

Cuidado: cualificación de restricciones

KKT son necesarias **si las restricciones activas son "regulares"**. La más usada: **LICQ** — los gradientes de las restricciones activas (∇h_i, ∇g_j activas) son **linealmente independientes**.

Contraejemplo
mín x s.a. $g(x) = x^2 \leq 0$.
La única x factible es 0, así que $x^* = 0$ es el mínimo. Pero $\nabla f = 1$ y $\nabla g(0) = 0$:

$$1 + \mu \cdot 0 = 0 \quad \text{no tiene solución.}$$

El mínimo existe y no cumple KKT, porque LICQ falla ($\nabla g = 0$).

- En la práctica: si los ∇g activos son dependientes (o se anulan), los multiplicadores pueden no existir o no ser únicos.
- Si todas las restricciones son lineales, KKT valen siempre (no se necesita LICQ).

Condiciones de segundo orden: ¿mínimo o máximo?

KKT no distinguen mínimos de máximos (P1 tenía ambos). Falta mirar la **curvatura** — pero solo *en las direcciones permitidas*:

$$d^\top \nabla^2_{xx} L(x^*, \lambda^*, \mu^*)\, d > 0 \quad \text{para toda } d \neq 0 \text{ "tangente"}, \qquad \nabla^2_{xx} L = \nabla^2 f + \sum \lambda_i \nabla^2 h_i + \sum$$

P1: $\nabla^2_{xx} L = 0 + \lambda \cdot 2I = 2\lambda I$
En $(-1,-1)$: $\lambda = \frac{1}{2} \Rightarrow \nabla^2 L = I$ definida positiva $\Rightarrow$ **mínimo**.
En $(1,1)$: $\lambda = -\frac{1}{2} \Rightarrow \nabla^2 L = -I$ definida negativa $\Rightarrow$ **máximo**.

- Aquí aparece la **Hessiana del Lagrangiano**, no la de f: la curvatura de las restricciones cuenta. (Será clave en SQP.)
- En problemas **convexos** no hace falta: KKT $\Rightarrow$ mínimo global.

Código: verificar KKT numéricamente

```python
def residuos_kkt(x, mu, grad, g, Jg):
    est  = np.linalg.norm(grad(x) + Jg(x).T @ mu)  # 1. estacionariedad
    fac  = max(0.0, np.max(g(x)))                  # 2. factibilidad primal
    dual = max(0.0, -np.min(mu))                   # 3. mu >= 0
    comp = np.max(np.abs(mu * g(x)))               # 4. complementariedad
    return est, fac, dual, comp

def multiplicadores_activos(x, grad, g, Jg, tol=1e-8):
    act = np.abs(g(x)) < tol                       # restricciones activas
    mu = np.zeros(len(act))                        # inactivas -> mu = 0
    mu[act] = np.linalg.lstsq(Jg(x)[act].T, -grad(x), rcond=None)[0]
    return mu                                      # grad f + Jg_A' mu = 0
```

Con la solución de SLSQP en P2: $x = (1, 1)$, $\mu = (0{,}6667,\ 0{,}6667)$, residuos $(5\times10^{-16},\ 2\times10^{-9},\ 0,\ 1{,}4\times10^{-9}) \Rightarrow$ KKT cumplidas.

Resumen de KKT

- **Igualdad**: $\nabla f \parallel \nabla h$, con λ de cualquier signo.
- **Desigualdad**: la restricción activa empuja hacia adentro: $\mu \geq 0$; la inactiva no empuja: $\mu g = 0$.
- Los **multiplicadores** son los precios sombra: $df^*/d\varepsilon = -\lambda^*$ (o $-\mu^*$).
- KKT son **necesarias** (con LICQ) y, en problemas **convexos**, **suficientes**; en general hay que revisar curvatura (Hessiana del Lagrangiano en el espacio tangente).

Puente con el resto de la clase

Resolver KKT a mano sirve para problemas chicos. En problemas reales, las KKT forman un **sistema no lineal con desigualdades** que hay que resolver *numéricamente*. Los cuatro métodos que siguen son distintas maneras de hacerlo: **QP**, **cutting plane**, **SQP** y **penalización**.

Programación cuadrática (QP)

¿Qué es un QP y por qué importa?

Problema de programación cuadrática

$$\min_{x \in \mathbb{R}^n} \ \tfrac{1}{2} x^\top H x + c^\top x \quad \text{s.a.} \quad Ax = b, \quad Gx \leq h$$

- **Objetivo cuadrático** (es el modelo de Newton de la Clase 4) + **restricciones lineales**.
- Aparece directamente: **mínimos cuadrados con restricciones**, portafolios, control (MPC), SVM...
- Y, sobre todo, aparece **como subproblema de SQP**: cada iteración de SQP resuelve un QP. Por eso lo estudiamos primero.

Convexidad

Si H es **semidefinida positiva**, el QP es **convexo**: las KKT son suficientes y todo mínimo local es global. Si H es indefinida, el problema puede ser muy difícil (en general NP-duro). Hoy suponemos H **definida positiva**.

Caso 1: solo igualdades → un sistema lineal

mín $\frac{1}{2}x^\top Hx + c^\top x$ s.a. $Ax = b$. Lagrangiano: $L = \frac{1}{2}x^\top Hx + c^\top x + \lambda^\top(Ax - b)$.

$$\nabla_x L = Hx + c + A^\top \lambda = 0, \qquad Ax = b$$

Son ecuaciones **lineales** en (x, λ). Juntas forman el **sistema KKT**:

$$\boxed{\begin{bmatrix} H & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} x \\ \lambda \end{bmatrix} = \begin{bmatrix} -c \\ b \end{bmatrix}}$$

- **No hay iteraciones**: se resuelve con `np.linalg.solve` (un solo paso).
- La matriz es simétrica **indefinida** ("punto silla"); es no singular si A tiene filas independientes y H es definida positiva en el núcleo de A.
- Con $H = 0$ y desigualdades sería un LP; con A vacía se reduce a Newton: $Hx = -c$.

Ejemplo: QP con una igualdad

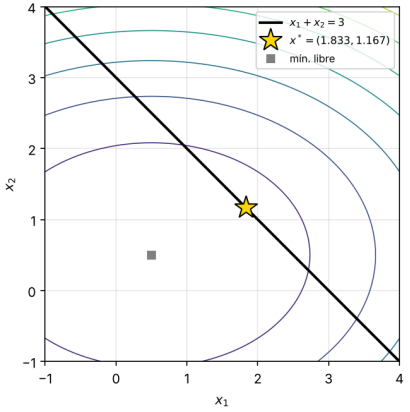
mín $(x_1 - \frac{1}{2})^2 + 2(x_2 - \frac{1}{2})^2$ s.a. $x_1 + x_2 = 3$

$H = \mathrm{diag}(2, 4)$, $c = (-1, -2)$, $A = [1\ 1]$, $b = 3$:

$$\begin{bmatrix} 2 & 0 & 1 \\ 0 & 4 & 1 \\ 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \lambda \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

Solución: $x_1 = \frac{11}{6}$, $x_2 = \frac{7}{6}$, $\lambda = -\frac{8}{3}$.

La curva de nivel **es tangente** a la recta en x^*: $\nabla f \parallel A^\top$.



Código: QP de igualdad

```
def qp_igualdad(H, c, A, b):
    """min 0.5 x'Hx + c'x  s.a. Ax=b  ->  [[H, A'],[A, 0]] [x; lam] = [-c; b]"
       ""
    n, m = H.shape[0], A.shape[0]
    K = np.block([[H, A.T], [A, np.zeros((m, m))]])
    sol = np.linalg.solve(K, np.concatenate([-c, b]))
    return sol[:n], sol[n:]          # x, lambda

# Ejemplo: min x'x  s.a.  x1 + 2 x2 + 3 x3 = 6
H = 2 * np.eye(3); c = np.zeros(3); A = np.array([[1., 2., 3.]]); b = np.array
    ([6.])
x, lam = qp_igualdad(H, c, A, b)
```

Resultado: $x = \frac{6}{14}(1,2,3) = (0{,}4286,\ 0{,}8571,\ 1{,}2857)$, $\lambda = -0{,}857$ — idéntico a `scipy` (`SLSQP`).

Caso 2: desigualdades — ¿cuáles están activas?

Si supiéramos qué restricciones están activas en el óptimo, **el problema sería un QP de igualdad** (un sistema lineal). La dificultad es saberlo.

Idea A: probar todas
Con m restricciones: 2^m combinaciones. Funciona ($m = 5 \Rightarrow 32$ casos y solo 1 cumple KKT), pero crece exponencialmente.

Idea B: método de conjunto activo
Adivinar, resolver y corregir: partir de un conjunto W de restricciones activas, resolver el QP de igualdad, y usar el resultado para **agregar o soltar** una restricción por iteración.

En la práctica se llega al óptimo en pocos cambios del conjunto W.

Método de conjunto activo: las reglas

Partimos de un punto **factible** x_k y un conjunto de restricciones activas W (tratadas como igualdades). Cada iteración:

1. **Resolver** el QP de igualdad "en el espacio de W": el paso p que minimiza el modelo con $G_W p = 0$ (un sistema KKT lineal).
2. **Si** $p \neq 0$: avanzar $x_{k+1} = x_k + \alpha p$ con el **mayor** $\alpha \leq 1$ que no viole ninguna restricción. Si algo bloquea ($\alpha < 1$), **agregar** esa restricción a W.
3. **Si** $p = 0$: x_k es el mejor punto con W activas. Calcular los **multiplicadores** μ_W de las restricciones de W.
4. **Si todos** $\mu \geq 0$: KKT cumplidas $\Rightarrow$ **óptimo**. Si algún $\mu < 0$, esa restricción **estorba**: **soltarla** (la más negativa) y repetir.

Paso máximo (restricciones $a_i^\top x \leq h_i$ fuera de W)

$$\alpha = \min\Big\{1,\ \min_{i \notin W,\ a_i^\top p > 0} \frac{h_i - a_i^\top x}{a_i^\top p}\Big\}$$ (solo frenan las que nos acercan al borde)

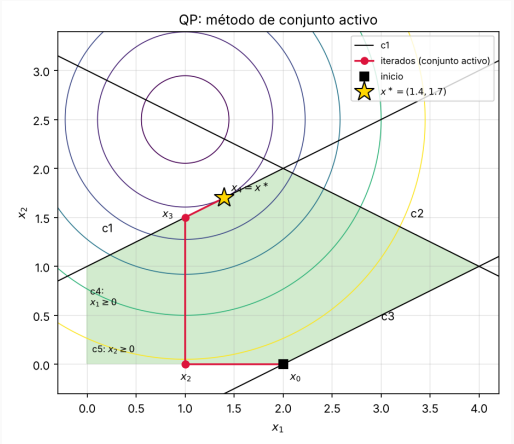
Ejemplo paso a paso: el problema

$$\text{mín } (x_1 - 1)^2 + (x_2 - 2{,}5)^2$$

$$\begin{aligned} c_1: &\quad x_1 - 2x_2 + 2 \geq 0 \\ c_2: &\quad -x_1 - 2x_2 + 6 \geq 0 \\ c_3: &\quad -x_1 + 2x_2 + 2 \geq 0 \\ c_4: &\quad x_1 \geq 0 \\ c_5: &\quad x_2 \geq 0 \end{aligned}$$

Inicio: $x_0 = (2, 0)$, con $W_0 = \{c_3, c_5\}$ activas (el punto está en la esquina de c_3 y c_5).

(En forma $Gx \leq h$ se escribe cambiando signos; ver el código.)



Ejemplo paso a paso: las iteraciones

it.	x	W	paso p y α / multiplicadores	acción
0	$(2,0)$	$\{c_3, c_5\}$	$p = 0$; $\mu_3 = -2$, $\mu_5 = -1$	**soltar** c_3 (más negativo)
1	$(2,0)$	$\{c_5\}$	$p = (-1,0)$, $\alpha = 1$	avanzar a $(1,0)$
2	$(1,0)$	$\{c_5\}$	$p = 0$; $\mu_5 = -5$	**soltar** c_5
3	$(1,0)$	$\emptyset$	$p = (0,2{,}5)$, $\alpha = 0{,}6$	choca con c_1: **agregar** c_1
4	$(1,1{,}5)$	$\{c_1\}$	$p = (0{,}4,0{,}2)$, $\alpha = 1$	avanzar a $(1{,}4,1{,}7)$
5	$(1{,}4,1{,}7)$	$\{c_1\}$	$p = 0$; $\mu_1 = 0{,}8 \geq 0$	**ÓPTIMO**

- $x^* = (1{,}4,\ 1{,}7)$, $f^* = 0{,}8$, con **una sola restricción activa** (c_1) y $\mu_1^* = 0{,}8$.
- Se movió por el **borde y por el interior**: soltó dos restricciones y agregó una. Total: 6 iteraciones (4 conjuntos activos distintos) frente a $2^5 = 32$ casos de la enumeración.

Código: núcleo del conjunto activo

```
for k in range(max_iter):
    g = H @ x + c                                   # gradiente del modelo en
        x
    p = qp_igualdad(H, g, G[W], np.zeros(len(W)))[0] if W else np.linalg.solve
        (H, -g)
    if np.linalg.norm(p) < tol:                     # x es minimo sobre W
        mu = np.linalg.solve(G[W] @ G[W].T, -G[W] @ g)   # g + G_W' mu = 0
        if np.all(mu >= -tol): break                # KKT cumplidas: OPTIMO
        W.remove(W[int(np.argmin(mu))])             # soltar la mas negativa
    else:
        alpha, blk = 1.0, None                      # paso maximo hasta el
            borde
        for i in range(G.shape[0]):
            if i not in W and G[i] @ p > 1e-12:
                a_i = (h[i] - G[i] @ x) / (G[i] @ p)
                if a_i < alpha: alpha, blk = a_i, i
        x = x + alpha * p
        if blk is not None: W.append(blk)           # agregar la restriccion
            bloqueante
```

Resultados y comparación con `scipy`

Método	x^*	f^*	iteraciones	Observación
Conjunto activo (manual)	(1,4, 1,7)	0.8	6 iteraciones	exacto (aritmética lineal)
`scipy SLSQP`	(1,4, 1,7)	0.8	3	SQP con BFGS
`scipy trust-constr`	(1,4, 1,7)	0.8	30	punto interior

- Para un QP, el conjunto activo da la solución **exacta** en un número **finito** de pasos (no se "converge", se *termina*).
- La solución coincide con la enumeración de KKT (un solo caso de 2^5 cumple todo: $S = \{c_1\}$, $\mu_1 = 0{,}8$).

Observación

Los μ del conjunto activo **son los multiplicadores de KKT**: el algoritmo *es* una búsqueda guiada del conjunto activo correcto, usando el signo de μ como brújula.

Detalles prácticos

- **Punto inicial factible**: si no se tiene, se resuelve primero un LP auxiliar (*fase I*) que encuentra un punto factible.
- H **no definida positiva**: el paso p puede no existir o no ser de descenso; se agrega un múltiplo de la identidad o se usan métodos específicos (como en el dogleg de la Clase 4, $B + \lambda I$).
- **Degeneración**: más de n restricciones activas en un punto (ciclado); se usan reglas anti-ciclado.
- **Alternativas**: métodos de **punto interior** (los usan los solvers modernos para QP grandes) y métodos duales.

Conexión con el miniproyecto

Ajustar $y = a\,e^{-bt} + c\,e^{-dt}$ con $b, d > 0$ y $b > d$ es un **problema de mínimos cuadrados con restricciones**. `scipy.optimize.least_squares(..., bounds=...)` (métodos `trf` y `dogbox`, de tipo trust region) lo resuelve; por dentro aparecen subproblemas cuadráticos con cotas como los de hoy.

Resumen de QP

- QP = objetivo cuadrático + restricciones lineales; convexo si $H \succeq 0$.
- **Solo igualdades**: un sistema lineal KKT $\begin{bmatrix} H & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} x \\ \lambda \end{bmatrix} = \begin{bmatrix} -c \\ b \end{bmatrix}$.
- **Con desigualdades**: método de conjunto activo = adivinar W, resolver, **agregar** la restricción que bloquea o **soltar** la de $\mu < 0$.
- Termina en un número finito de pasos con la solución exacta.

Siguiente pregunta
Si el problema real **no** es un QP (funciones no lineales), ¿cómo usamos esto? Dos caminos: **linealizar todo** (cutting plane $\rightarrow$ LP) o **cuadratizar** el objetivo y linealizar las restricciones (SQP $\rightarrow$ QP).

Cutting plane (planos de corte)

La idea: reemplazar lo difícil por algo lineal

- Problema original: f y g_j **no lineales**. Sabemos resolver muy bien problemas **lineales** (LP: simplex, puntos interiores).
- **Idea**: aproximar f y g_j por sus **rectas/planos tangentes** (linearización) y resolver el LP resultante.
- Un solo LP no basta: la aproximación es buena solo cerca del punto de tangencia. Entonces se **agrega un nuevo plano tangente** en cada solución encontrada y se resuelve de nuevo.

Por qué se llama "planos de corte"
Cada plano tangente **corta** una parte de la región donde está permitida la solución del LP, y la va ajustando a la región real. Es el método de **Kelley** (1960), para problemas **convexos**.

Linearizar: el plano tangente es una cota

Para f **convexa**, la tangente en x_k queda **por debajo** de la función:

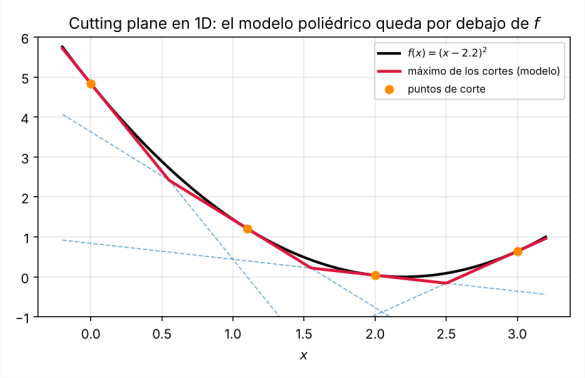
$$f(x) \;\geq\; f(x_k) + \nabla f(x_k)^\top (x - x_k) \quad \forall x.$$

Igual para cada g_j convexa: $g_j(x) \geq g_j(x_k) + \nabla g_j(x_k)^\top (x - x_k)$.

Consecuencia en las restricciones
Si $g_j(x) \leq 0$, entonces la linearización también es ≤ 0. La región **lineal** $\{g_j(x_k) + \nabla g_j(x_k)^\top (x - x_k) \leq 0\}$ **contiene** a la región factible real: es una aproximación **por fuera**.

Consecuencia en el objetivo
El modelo $\max_k \{f(x_k) + \nabla f(x_k)^\top (x - x_k)\}$ queda **por debajo** de f: el LP nunca puede "pasarse" del óptimo real.

Cutting plane en 1D: el modelo poliédrico



- Cada punto de corte aporta una **recta tangente** (líneas punteadas).
- El **modelo** es el máximo de todas las rectas (rojo): una función lineal por tramos **por debajo** de f.
- Cuantos más cortes, mejor lo aproxima; los cortes nuevos se agregan **donde el LP cree que está el mínimo**, así que el modelo se afina justamente donde importa.

El truco del epígrafo

Para que **todo** sea lineal (incluso el objetivo) se introduce una variable t:

$$\min_{x} f(x) \quad \Longleftrightarrow \quad \min_{x,t} \; t \quad \text{s.a.} \quad f(x) \leq t.$$

LP de la iteración k (con los cortes de las iteraciones $0, \ldots, k$)

$$\begin{aligned} \min_{x,t} \quad & t \\ \text{s.a.} \quad & f(x_i) + \nabla f(x_i)^\top (x - x_i) \leq t, && i = 0, \ldots, k \\ & g_j(x_i) + \nabla g_j(x_i)^\top (x - x_i) \leq 0, && i = 0, \ldots, k, \;\; j = 1, \ldots, p \\ & -B \leq x \leq B && \text{(caja, para que el LP esté acotado)} \end{aligned}$$

- Es un **LP** con $n + 1$ variables: se resuelve con `linprog`. Su solución (x_{k+1}, t_{k+1}) define el siguiente punto de corte.

Algoritmo de cutting plane (Kelley)

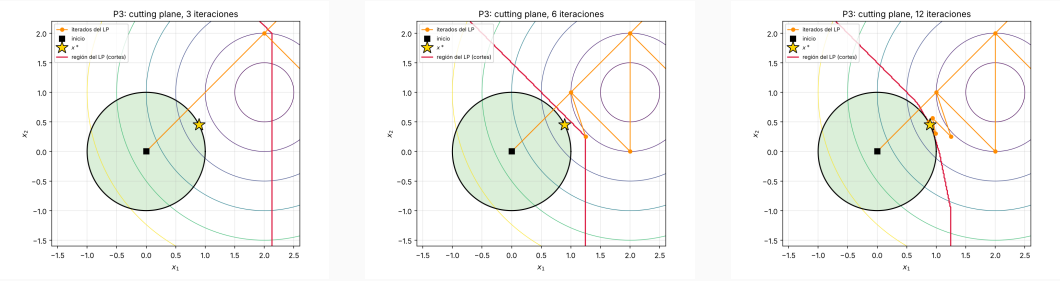
1. Elegir x_0 y una caja $[-B, B]^n$ que contenga al óptimo.
2. **Agregar cortes** en x_k: uno para f y uno por cada g_j (necesita $f, \nabla f, g, \nabla g$ en x_k).
3. **Resolver el LP** con todos los cortes acumulados $\to (x_{k+1}, t_{k+1})$.
4. **Parar** si x_{k+1} es casi factible ($\max g_j \le \text{tol}$) y la brecha $f(x_{k+1}) - t_{k+1} \le \text{tol}$. Si no, volver a (2).

Certificado de optimalidad gratis

Como el LP es una relajación, $t_{k+1} \le f^*$ es una **cota inferior** en cada iteración. Si x_{k+1} es factible, $f(x_{k+1}) \ge f^*$ es una cota superior. La **brecha** $f(x_{k+1}) - t_{k+1}$ dice qué tan cerca estamos — sin conocer f^*.

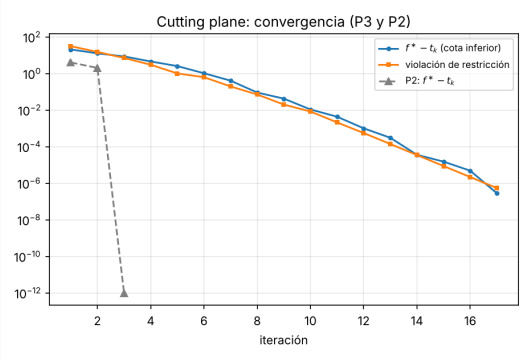
Los iterados x_k del LP **pueden ser infactibles** para el problema original (la linearización deja pasar de más): solo al final lo son.

Ejemplo P3 (restricción curva): cómo se cierra el poliedro



Izquierda a derecha: 3, 6 y 12 iteraciones. La curva roja es la **región del LP** (intersección de los semiplanos de los cortes de g): al principio es enorme; luego se **envuelve** al disco como un polígono circunscrito. Los puntos naranjas (soluciones del LP) **se acercan a x^* desde afuera** del disco.

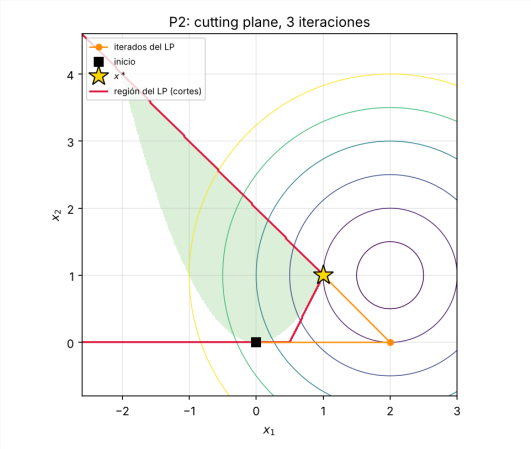
P3: convergencia del cutting plane



k	x_{k+1}	t_{k+1}	$f(x_{k+1})$	viol.
0	(4, 4)	-19	13	31
2	(2, 2)	-7	1	7
4	(1, 1)	-1	1	1
6	(0,94, 0,56)	1.125	1.320	0.20
8	(0,92, 0,42)	1.486	1.506	0.020
9	(0,87, 0,50)	1.517	1.525	0.008
14	(0,895, 0,446)	1.527849	1.527858	9×10^{-6}
16	(0,8947, 0,4466)	1.527864	1.527864	5×10^{-7}

17 LPs (34 cortes), con $f^* = 1{,}527864$: la cota inferior t_k **sube monótonamente** hacia f^*.

Contraste: P2 se resuelve en 3 iteraciones



- En P2 el LP cae **exactamente** sobre $(1, 1)$ en la 2.ª iteración: allí se cruzan la restricción g_2 (lineal, su corte es exacto) y la "punta" que forman dos tangentes de f.
- Ese punto es un **vértice** del poliedro de cortes. En la 3.ª iteración, con las tangentes en $(1, 1)$, t sube de -1 a $1 = f^*$ y **se certifica** el óptimo.
- **No es el caso típico**: cuando el óptimo está sobre una frontera curva (P3), el método tiene que "circunscribir" la curva con muchos cortes y converge lento.

Código: cutting plane con `linprog`

```
def cutting_plane(f, grad, g, Jg, x0, B=4.0, tol=1e-6, max_iter=60):
    x = np.array(x0, float);  A_ub, b_ub = [], []
    for k in range(max_iter):
        fk, gk = f(x), grad(x)
        A_ub.append(np.r_[gk, -1.0]);  b_ub.append(gk @ x - fk)        # corte de f
        for gi, Ji in zip(g(x), Jg(x)):
            A_ub.append(np.r_[Ji, 0.0]);  b_ub.append(Ji @ x - gi)     # corte de g_i
        c = np.zeros(len(x) + 1);  c[-1] = 1.0                         # min t
        res = linprog(c, A_ub=np.array(A_ub), b_ub=np.array(b_ub),
                      bounds=[(-B, B)] * len(x) + [(None, None)], method="highs")
        x, t = res.x[:-1], res.x[-1]
        if max(0.0, np.max(g(x))) < tol and f(x) - t < tol:            # factible y brecha cerra
            break
    return x
```

Resultados: P3 $\to x = (0{,}89471, 0{,}44664)$ con 17 LPs ($x^* = (0{,}89443, 0{,}44721)$); `SLSQP`: 17 iteraciones. P2 $\to$ 3 LPs.

Ventajas y límites del cutting plane

Ventajas

- Muy simple: solo un solver de **LP** y gradientes.
- Maneja **restricciones no lineales convexas**.
- Da **cotas inferiores** y un certificado de optimalidad.
- Funciona con funciones no diferenciables (usando subgradientes).

Límites

- **Lento**: aproxima una curva con segmentos rectos; el costo crece con la dimensión.
- **El LP crece**: cada iteración agrega cortes (memoria y tiempo).
- Necesita la **caja** para acotar el LP; los iterados **pueden ser infactibles**.
- Exige **convexidad**: sin ella, la tangente ya no es cota y puede cortar al propio óptimo.

Resumen de cutting plane

- Convertir el problema en una **secuencia de LPs** con planos tangentes (cortes) de f y g_j, usando el truco del epígrafo.
- Los cortes **nunca eliminan el óptimo** (si el problema es convexo) y el LP da una **cota inferior** que sube.
- Casos con vértice óptimo: muy rápido (P2). Frontera curva: lento (P3, 17 LPs).

¿Podemos hacerlo mejor que lineal?
Sí: si el modelo del objetivo **conserva la curvatura** (cuadrático) y solo las restricciones se linearizan, el LP se convierte en un **QP** y la convergencia pasa de lenta a **cuadrática**. Eso es la **programación cuadrática secuencial (SQP)**.

Programación cuadrática secuencial (SQP)

Recordatorio: Newton = resolver un modelo cuadrático

- Clase 4, sin restricciones: en x_k se construye el modelo $m(p) = f_k + \nabla f_k^\top p + \frac{1}{2} p^\top \nabla^2 f_k \, p$ y el paso de Newton es su minimizador: $\nabla^2 f_k \, p = -\nabla f_k$.
- Con restricciones, el equivalente natural es: **modelo cuadrático del objetivo** + **restricciones linearizadas**. Eso es exactamente un **QP**.

Idea de SQP

En cada iteración: **resolver un QP** que aproxima el problema en x_k, y dar el paso que ese QP propone.

- Lo bonito: SQP **no** es un truco inventado; es **el método de Newton aplicado a las condiciones KKT**. Lo vemos a continuación.

SQP = Newton sobre las condiciones KKT (caso igualdades)

Para mín f s.a. $h(x) = 0$ las KKT son un sistema de $n + m$ ecuaciones:

$$F(x, \lambda) = \begin{bmatrix} \nabla f(x) + J_h(x)^\top \lambda \\ h(x) \end{bmatrix} = 0 \qquad (J_h = \text{jacobiana de } h)$$

Newton para $F = 0$ (con $W = \nabla^2_{xx} L$ la Hessiana del Lagrangiano):

$$\begin{bmatrix} W & J_h^\top \\ J_h & 0 \end{bmatrix} \begin{bmatrix} p \\ \Delta\lambda \end{bmatrix} = - \begin{bmatrix} \nabla_x L \\ h \end{bmatrix} \iff \boxed{\begin{bmatrix} W & J_h^\top \\ J_h & 0 \end{bmatrix} \begin{bmatrix} p \\ \lambda_+ \end{bmatrix} = \begin{bmatrix} -\nabla f \\ -h \end{bmatrix}}, \quad \lambda_+ = \lambda + \Delta\lambda$$

¡Es el sistema KKT de un QP!

$$\min_p \ \nabla f^\top p + \tfrac{1}{2} p^\top W p \qquad \text{s.a.} \qquad J_h\, p + h = 0$$

(compare con la diapositiva "QP de igualdad": $H \to W$, $c \to \nabla f$, $A \to J_h$, $b \to -h$.) Resolver el QP **es** dar un paso de Newton sobre KKT, y su multiplicador λ_+ es la nueva estimación de λ.

El subproblema QP completo (con desigualdades)

En x_k (con multiplicadores actuales λ_k, μ_k):

$$\begin{aligned} \min_{p} \quad & \nabla f(x_k)^\top p + \tfrac{1}{2}\, p^\top W_k\, p \\ \text{s.a.} \quad & h(x_k) + J_h(x_k)\, p = 0 \\ & g(x_k) + J_g(x_k)\, p \le 0 \end{aligned} \qquad W_k = \nabla^2 f_k + \sum_i \lambda_i \nabla^2 h_i + \sum_j \mu_j \nabla^2 g_j$$

- Las restricciones se **linearizan** (como en cutting plane); el objetivo se **cuadratiza** (como en Newton). Se resuelve con el método de conjunto activo.
- Del QP salen el paso p_k **y** los multiplicadores λ_{k+1}, μ_{k+1} (**gratis**, son los de KKT del QP).
- El conjunto activo del QP **predice** qué restricciones estarán activas en el óptimo real.

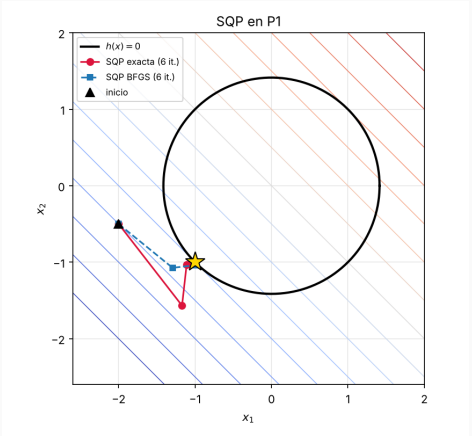
¿Por qué W es la Hessiana del Lagrangiano y no la de f?

P1: $f = x_1 + x_2$ es *lineal*: $\nabla^2 f = 0$.

- Si usáramos $W = \nabla^2 f = 0$, el QP sería un **LP** sobre la restricción linearizada (una recta): **sin mínimo** (se escapa al infinito).
- Con $W = \nabla^2_{xx} L = \lambda \cdot 2I$, el QP "ve" la **curvatura de la circunferencia**: tiene mínimo y apunta bien al óptimo.

Moraleja

La curvatura de las restricciones **también cuenta** en el óptimo (ya lo vimos en las condiciones de 2.º orden). Por eso SQP modela el **Lagrangiano**, no solo f.



Algoritmo SQP básico

1. Dado x_k y multiplicadores λ_k, μ_k: calcular $f, \nabla f, h, J_h, g, J_g$ y W_k (exacta o aproximada).
2. **Resolver el QP** $\rightarrow$ paso p_k y nuevos multiplicadores $\hat{\lambda}, \hat{\mu}$.
3. **Elegir** $\alpha_k \in (0, 1]$ por búsqueda lineal sobre una **función de mérito** (siguiente diapositiva).
4. Actualizar: $x_{k+1} = x_k + \alpha_k p_k$, $\lambda_{k+1} = \lambda_k + \alpha_k(\hat{\lambda} - \lambda_k)$, igual para μ.
5. Parar si $\max(\|\nabla_x L\|_\infty,\ \text{violación},\ \max|\mu_j g_j|) < \text{tol}$ (**el criterio es KKT**).

Comparación con las clases anteriores

Es el **mismo esqueleto** de Newton con line search (Clase 4): un paso de un modelo cuadrático + control del tamaño del paso. La novedad: el modelo es un QP con restricciones y hace falta una función de mérito para decidir si el paso mejora.

Globalización: la función de mérito ℓ_1

- Lejos de la solución, Newton puede dar pasos que **bajan** f pero **violan** más las restricciones (o al revés). Sin restricciones, el criterio era "f baja". Con restricciones hay **dos objetivos en conflicto**.
- Solución: una función que mide **ambos**:

$$\phi(x;\nu) = f(x) + \nu\Big(\sum_i |h_i(x)| + \sum_j \max\{0, g_j(x)\}\Big), \qquad \nu > 0$$

Búsqueda lineal (Armijo)
Aceptar α si

$$\phi(x + \alpha p) \le \phi(x) + \eta\,\alpha\,D,$$

con $\eta = 10^{-4}$, $D = \nabla f^{\top} p - \nu \cdot \text{viol}(x)$, y retroceder $\alpha \leftarrow \alpha/2$ si no.

Elección de ν
Debe ser **mayor que los multiplicadores** ($\nu > \|\lambda\|_\infty$, máx μ_j): así ϕ es una **penalización exacta** (su mínimo *es* el de la restricción) y el paso p de SQP es **dirección de descenso** de ϕ.

Hessiana exacta o BFGS

Hessiana exacta de L
Convergencia **cuadrática**, pero necesita segundas derivadas de f **y de las restricciones**. Puede ser indefinida $\Rightarrow$ se la corrige (sumar τI) para que el QP sea convexo.

Aproximación BFGS de W
Como en la Clase 4, se actualiza con $s = x_{k+1} - x_k$ y $y = \nabla_x L(x_{k+1}, \lambda_+) - \nabla_x L(x_k, \lambda_+)$.
Convergencia **superlineal**, **solo primeras derivadas**.

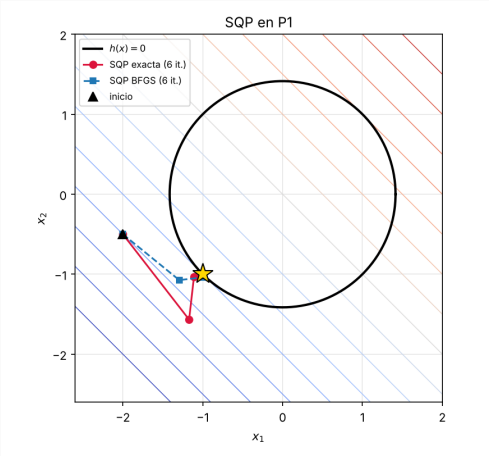
Amortiguamiento de Powell
La Hessiana del Lagrangiano **no** es definida positiva en general, así que $s^\top y$ puede ser ≤ 0 y romper BFGS. Powell reemplaza y por $r = \theta y + (1-\theta)Bs$ (con $\theta \in [0,1]$ elegido para que $s^\top r \geq 0{,}2\, s^\top Bs$), que **mantiene B definida positiva**.

Ejemplo P1: iteraciones de SQP (Hessiana exacta)

$x_0 = (-2,\ -0{,}5)$, λ_0 por mínimos cuadrados:

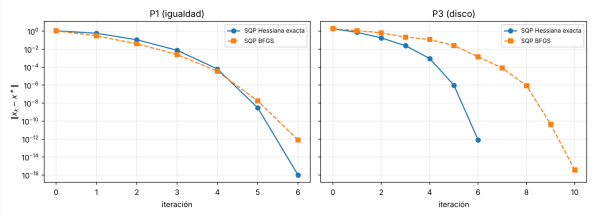
k	x_k	$f(x_k)$	viol.
0	$(-2,\ -0{,}5)$	$-2{,}5$	2.25
1	$(-1{,}1706,\ -1{,}5676)$	$-2{,}738$	1.83
2	$(-1{,}1097,\ -1{,}0301)$	$-2{,}140$	0.29
3	$(-0{,}9988,\ -1{,}0075)$	$-2{,}0064$	$1{,}3\times10^{-2}$
4	$(-1{,}00006,\ -0{,}99997)$	$-2{,}00003$	$5{,}8\times10^{-5}$
5	$(-1,\ -1)$	-2	4×10^{-9}

Converge a $x^* = (-1, -1)$, $\lambda^* = 0{,}5$.



Ambas versiones (exacta y BFGS) llegan en 6 iteraciones; el punto inicial es infactible y las iteraciones **van acercándose a la circunferencia** mientras f sube hacia -2.

Tasa de convergencia: cuadrática vs. superlineal

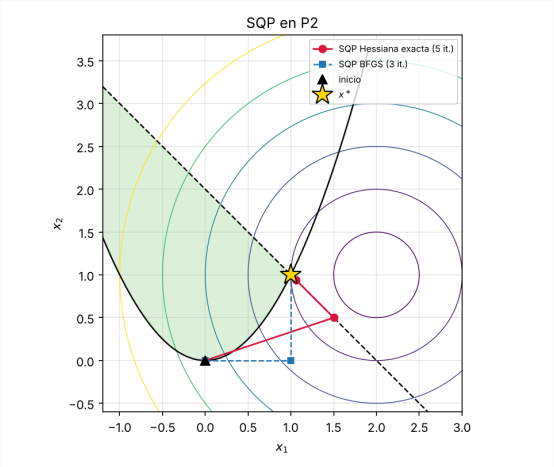


Error $\|x_k - x^*\|$ en P3 (disco):

k	exacta	BFGS
1	$7{,}5\times10^{-1}$	$1{,}2$
2	$1{,}8\times10^{-1}$	$6{,}6\times10^{-1}$
3	$2{,}4\times10^{-2}$	$2{,}2\times10^{-1}$
4	$9{,}4\times10^{-4}$	$1{,}2\times10^{-1}$
5	$9{,}9\times10^{-7}$	$2{,}5\times10^{-2}$
6	$8{,}3\times10^{-13}$	$1{,}4\times10^{-3}$

Exacta: el error se *eleva al cuadrado* ($10^{-4} \to 10^{-7} \to 10^{-13}$).
BFGS: más iteraciones pero cada una más barata.

Ejemplo P2: SQP y el conjunto activo del QP



k	x_k	error	activas en el
0	(0, 0)	1.41	g_2
1	(1,5, 0,5)	0.71	g_1, g_2
2	(1,0625, 0,9375)	$8{,}8\times10^{-2}$	g_1, g_2
3	(1,00125, 0,99875)	$1{,}8\times10^{-3}$	g_1, g_2
4	(1,000001, 0,999999)	$1{,}4\times10^{-6}$	g_1, g_2
5	(1, 1)	$\sim 10^{-12}$	–

- $\alpha = 1$ en todas las iteraciones.
- Desde la 1.ª iteración el QP **identifica las dos restricciones activas**; luego es Newton puro sobre KKT.
- Multiplicadores finales: $\mu = (\frac{2}{3}, \frac{2}{3})$ ✓.

Código: el QP por enumeración de conjuntos activos

```
def resolver_qp(W, gf, Aeq, ceq, Ain, cin, tol=1e-9):
    """min gf'p + 0.5 p'Wp  s.a. Aeq p + ceq = 0,  Ain p + cin <= 0."""
    n, me, mi = len(gf), len(ceq), len(cin)
    for r in range(mi + 1):                          # prueba subconjuntos S de desigualdades activ
        for S in itertools.combinations(range(mi), r):
            S = list(S)
            A = np.vstack([Aeq, Ain[S]]);  rhs = np.concatenate([-ceq, -cin[S]])
            K = np.block([[W, A.T], [A, np.zeros((len(rhs), len(rhs)))]])
            sol = np.linalg.solve(K, np.concatenate([-gf, rhs]))   # sistema KKT de igualdad
            p, nu = sol[:n], sol[n:]
            lam, mu_S = nu[:me], nu[me:]
            if mi and np.any(Ain @ p + cin > tol):  continue       # viola otra desigualdad
            if len(S) and np.any(mu_S < -tol):      continue       # mu < 0: no es KKT
            mu = np.zeros(mi);  mu[S] = mu_S
            return p, lam, mu, S
```

Exacto y claro para pocas desigualdades; en la práctica se usa el conjunto activo de la sección de QP.

Código: bucle principal de SQP

```
def sqp(prob, x0, modo="exacta", tol=1e-9, max_iter=50):
    x = np.array(x0, float);  B = np.eye(len(x));  nu_pen = 1.0
    lam = np.linalg.lstsq(Jh(x).T, -grad(x), rcond=None)[0] if me else np.zeros(0);  mu = np.zer
    for k in range(max_iter):
        kkt = max(np.max(np.abs(gradL(x, lam, mu))), viol(x), np.max(np.abs(mu * g(x))))
        if kkt < tol:  break                                    # criterio de parada: KKT
        W = hessL(x, lam, mu) if modo == "exacta" else B        # Hessiana del Lagrangiano
        W = W + max(0.0, 1e-3 - np.linalg.eigvalsh(W).min()) * np.eye(len(x))   # W definida pos
        p, lam_n, mu_n, S = resolver_qp(W, grad(x), Jh(x), h(x), Jg(x), g(x))   # QP
        nu_pen = max(nu_pen, 1.1 * np.max(np.abs(np.r_[lam_n, mu_n])) + 1e-3)   # penalidad de m
        D = grad(x) @ p - nu_pen * viol(x);  alpha = 1.0
        while phi(x + alpha*p, nu_pen) > phi(x, nu_pen) + 1e-4*alpha*D:  alpha /= 2   # Armijo
        x_new = x + alpha * p
        lam, mu = lam + alpha*(lam_n - lam), mu + alpha*(mu_n - mu)
        if modo == "bfgs": B = bfgs_powell(B, x_new - x, gradL(x_new, lam, mu) - gradL(x, lam, m
        x = x_new
    return x, lam, mu
```

(Versión resumida; el archivo completo está en `scripts/13_sqp.py`.)

Resultados: P3 (convexo) y P4 (Rosenbrock en el disco, no convexo)

P3 — $x^* = (0{,}8944, 0{,}4472)$

Método	iter.	eval. f	eval. ∇f	error
SQP exacta	6	19	13	$< 10^{-12}$
SQP BFGS	7	30	29	10^{-16}
`SLSQP`	9	11	9	3×10^{-15}

P4 — $x^* = (0{,}7864, 0{,}6177)$, $f^* = 0{,}0457$

Método	iter.	eval. f	eval. ∇f	error
SQP exacta	6	41	13	5×10^{-9}
SQP BFGS	22	97	89	5×10^{-9}
`SLSQP`	17	23	17	5×10^{-9}

- El SQP manual coincide con `scipy SLSQP` (que *es* un SQP con BFGS y función de mérito ℓ_1) en **la solución y en el orden de magnitud de iteraciones**.
- P4 es **no convexo** (Rosenbrock): SQP sigue funcionando porque solo necesita **puntos KKT** locales; cutting plane **no** aplicaría.
- Conteos aproximados (cada evaluación del mérito ϕ cuenta como una evaluación de f).

Dos advertencias honestas

SQP converge a *puntos KKT*, no necesariamente al mínimo
En P1, $(1,1)$ es un **máximo** que también cumple KKT. Partiendo de $x_0 = (1{,}5, 1)$ (cerca de él) nuestro SQP llega al mínimo, pero tarda **49 iteraciones** con Hessiana exacta (16 con BFGS): el método "duda" cerca del máximo, donde $W = 2\lambda I$ es negativa y debe ser corregida. Regla práctica: arrancar lejos de puntos KKT no deseados y **verificar** el resultado.

Efecto Maratos (comentario)
Cerca de la solución, el paso completo de Newton puede **aumentar** la función de mérito aunque se acerque a x^* (la linearización de restricciones curvas introduce un error de 2.º orden). El line search lo rechazaría y se perdería la velocidad. Remedios: **corrección de 2.º orden**, filtros, o *watchdog*. En nuestros ejemplos no apareció: $\alpha = 1$ en las iteraciones finales.

SQP vs. cutting plane: qué cambia

	Cutting plane	**SQP**
Modelo del objetivo	lineal (epígrafo)	**cuadrático** (Hessiana de L)
Restricciones	linearizadas	linearizadas
Subproblema	LP	QP
Convexidad	**necesaria**	no necesaria (puntos KKT)
Convergencia	lenta (P3: 17 LPs)	**cuadrática/superlineal** (P3: 6 QPs)
Iterados	suelen ser infactibles	suelen ser infactibles
Cotas inferiores	sí	no

En una frase

Conservar la **curvatura** del Lagrangiano cuesta resolver un QP en vez de un LP, y a cambio la convergencia es de otro orden.

En `scipy`: SQP en la práctica

```
from scipy.optimize import minimize
cons = [{"type": "ineq", "fun": lambda x: -g(x), "jac": lambda x: -Jg(x)}]   #
     scipy: c(x) >= 0
res = minimize(f, x0, jac=grad, constraints=cons, method="SLSQP")
```

- `SLSQP` (*Sequential Least Squares Programming*, Kraft 1988): SQP con **BFGS** y función de mérito ℓ_1; el QP se resuelve como un problema de mínimos cuadrados.
- Nota de convención: `scipy` espera restricciones $c(x) \geq 0$ (`type: "ineq"`), por eso se pasa $-g$.
- `trust-constr`: usa **SQP con regiones de confianza** para igualdades y **punto interior** para desigualdades (relacionado con la sección siguiente).
- Resultado en P2: `SLSQP` $x = (1, 1)$ en 10 iteraciones (nit), nuestro SQP exacto en 5.

Resumen de SQP

- **SQP = Newton sobre KKT**: en cada iteración se resuelve un QP (modelo cuadrático del **Lagrangiano** + restricciones linearizadas).
- El QP entrega el paso **y** los multiplicadores; el conjunto activo del QP **predice** el del problema real.
- **Función de mérito** ℓ_1 + búsqueda lineal para garantizar progreso global; $\nu > \|\lambda\|_\infty$.
- **Hessiana exacta**: cuadrática. **BFGS** (con amortiguamiento de Powell): superlineal, sin segundas derivadas.
- Funciona en problemas **no convexos** y es la base de `SLSQP`; converge a **puntos KKT**.

Otra filosofía

En vez de resolver un subproblema con restricciones lineales en cada paso, **eliminar las restricciones** penalizando su violación: métodos de **penalización** (exterior e interior).

Métodos de penalización: exterior e interior

La idea: convertir el problema con restricciones en uno sin restricciones

- Ya sabemos resolver problemas **sin** restricciones (Clase 4: Newton, BFGS, gradiente conjugado). ¿Podemos reutilizarlos?
- **Sí**: añadir al objetivo un **castigo** por violar (o por acercarse a violar) las restricciones y resolver **una sucesión** de problemas sin restricciones.

Penalización exterior
Se permite **violar** las restricciones, pero se **castiga**. Iterados **infactibles** que se acercan a la región factible desde afuera cuando $\rho \to \infty$.

Barrera interior
Se **prohíbe** salir: una barrera que $\to \infty$ en la frontera. Iterados **factibles** que se acercan a la frontera desde adentro cuando $\mu \to 0$.

Esqueleto común
Resolver mín $F_\theta(x)$ para una sucesión de parámetros $\theta_1, \theta_2, \ldots$ cada vez más "duros", usando la solución anterior como punto inicial (**warm start**).

Penalización exterior (cuadrática)

Función penalizada

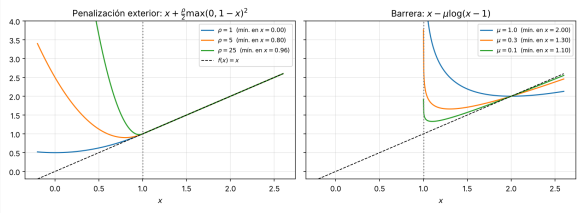
$$Q(x;\rho) = f(x) + \frac{\rho}{2}\Big(\sum_i h_i(x)^2 + \sum_j \max\{0,\, g_j(x)\}^2\Big), \qquad \rho > 0$$

- Dentro de la región factible la penalidad es **cero**; fuera crece como el cuadrado de la violación.
- $\max\{0, g\}^2$ es derivable una vez (la Hessiana salta en $g = 0$), suficiente para Newton con line search en la práctica.
- Gradiente: $\nabla Q = \nabla f + \rho(\sum_i h_i \nabla h_i + \sum_j \max\{0, g_j\}\nabla g_j)$.

Algoritmo

Elegir ρ_1 pequeño y x_0. Para $k = 1, 2, \ldots$: resolver mín $Q(x;\rho_k)$ partiendo de $x_{k-1} \to x_k$; si la violación es menor que la tolerancia, parar; si no, $\rho_{k+1} = 10\,\rho_k$.

Ejemplo 1D: mín x s.a. $x \geq 1$ (es decir $g = 1 - x \leq 0$)



Exterior: $Q = x + \frac{\rho}{2}\,\text{máx}\{0, 1-x\}^2$.
$Q' = 1 - \rho(1-x) = 0 \Rightarrow x_\rho = 1 - \frac{1}{\rho}$.
El mínimo queda **fuera** (infactible) por $1/\rho$ y converge a $x^* = 1$ cuando $\rho \to \infty$.

Barrera (ver más adelante):
$B = x - \mu\log(x-1)$.
$B' = 1 - \frac{\mu}{x-1} = 0 \Rightarrow x_\mu = 1 + \mu$.
El mínimo queda **dentro** (factible) por μ.

Penalización exterior y multiplicadores

En el mínimo x_ρ de Q: $\nabla f + \rho \sum_j g_j^+ \nabla g_j = 0$.
Compárese con KKT: $\nabla f + \sum_j \mu_j \nabla g_j = 0$.

Estimador de multiplicadores

$$\hat{\mu}_j = \rho \, \max\{0, g_j(x_\rho)\}, \qquad \hat{\lambda}_i = \rho \, h_i(x_\rho)$$

Convergen a μ^*, λ^* con error $O(1/\rho)$.

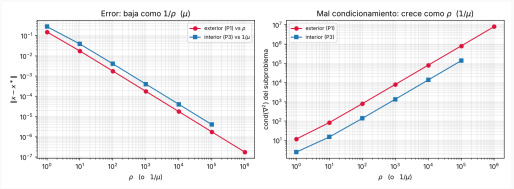
- "Violación $\times\rho$" = multiplicador: **cuánto cuesta** la restricción.
- Gratis: no hay que resolver nada extra.

P2 (exterior), $\mu^* = (\frac{2}{3}, \frac{2}{3})$

ρ	x_1	x_2	f	viol.	$\hat{\mu}_1$	$\hat{\mu}_2$	cond.
10^0	1.24594	1.07661	0.57447	4.8e-01	0.476	0.323	2.9e+00
10^1	1.04053	1.01917	0.92096	6.4e-02	0.635	0.597	3.2e+00
10^2	1.00440	1.00219	0.99122	6.6e-03	0.663	0.659	3.1e+00
10^3	1.00044	1.00022	0.99911	6.7e-04	0.666	0.666	3.1e+00
10^4	1.00004	1.00002	0.99991	6.7e-05	0.667	0.667	3.1e+00
10^5	1.00000	1.00000	0.99999	6.7e-06	0.667	0.667	3.1e+00
10^6	1.00000	1.00000	1.00000	6.7e-07	0.667	0.667	3.1e+00

El precio: mal condicionamiento cuando $\rho \to \infty$

- $\nabla^2 Q = \nabla^2 f + \rho \sum_j g_j^+ \nabla^2 g_j + \rho \sum_{j \text{ activas}} \nabla g_j \nabla g_j^\top$.
- El último término tiene **rango** $\leq$ **nº de restricciones activas** y tamaño $\propto \rho$: crea **valles muy empinados** en esas direcciones y planos en las demás.
- $\text{cond}(\nabla^2 Q) \sim O(\rho) \Rightarrow$ Newton/BFGS se vuelven inestables o lentos para ρ enorme.



Por eso se usa una sucesión
Con ρ moderado el subproblema es fácil y da un buen x_0 para el siguiente: **warm start**.

Condicionamiento y error: P1 (exterior) y P3 (interior)

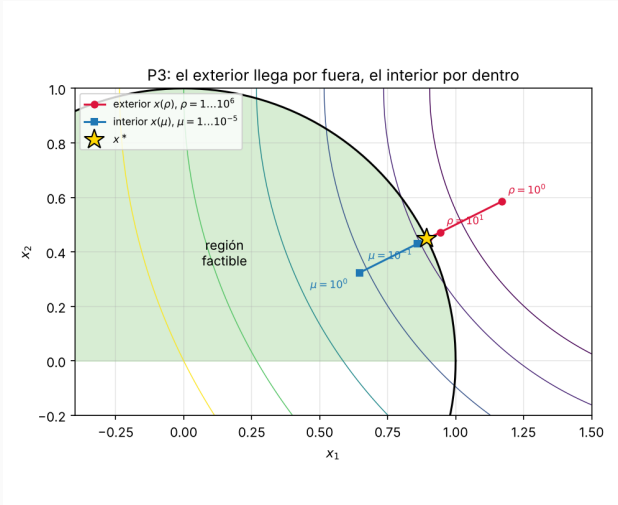
ρ o $1/\mu$	cond. ext. (P1)	cond. int. (P3)	err. ext. (P1)	err. int. (P3)
10^0	1.2e+01	2.5e+00	1.5e-01	2.8e-01
10^1	8.4e+01	1.5e+01	1.7e-02	3.8e-02
10^2	8.0e+02	1.4e+02	1.8e-03	4.0e-03
10^3	8.0e+03	1.4e+03	1.8e-04	4.0e-04
10^4	8.0e+04	1.4e+04	1.8e-05	4.0e-05
10^5	8.0e+05	1.4e+05	1.8e-06	4.0e-06

- **Error** $\propto 1/\rho$ (o $\propto \mu$): cada factor 10 en el parámetro da **un decimal** más.
- **Condición** $\propto \rho$ (o $\propto 1/\mu$): cada decimal **cuesta** un factor 10 en la condición de la Hessiana.

¿Siempre pasa?

No. En P2 la solución es un **vértice** (2 restricciones activas en 2 variables) y la condición **no crece** (queda en ≈ 3, tabla anterior): el mal condicionamiento aparece cuando hay **menos restricciones activas que variables**, como en P1 y P3.

Trayectoria de la penalización exterior (P3)



- Exterior: x_ρ parte **fuera** del disco y se acerca por afuera:
  $\rho = 1 \to (1{,}17, 0{,}58)$,
  $\rho = 10 \to (0{,}94, 0{,}47)$,
  $\cdots \to (0{,}8944, 0{,}4472)$.
- Interior: x_μ parte **dentro** y se acerca por adentro.
- Las dos trayectorias **rodean** la solución x^* en la frontera, una por cada lado.
- Estimador exterior:
  $\hat{\mu} = 1{,}22, 1{,}235, 1{,}2359 \to \mu^* = \sqrt{5} - 1 = 1{,}2361$.

Barrera interior (logarítmica)

Función barrera (solo desigualdades $g_j \le 0$)

$$B(x;\mu) = f(x) - \mu \sum_j \log(-g_j(x)), \qquad \mu > 0, \quad g_j(x) < 0 \text{ (estrictamente factible)}$$

- Si $g_j \to 0^-$, entonces $-\log(-g_j) \to +\infty$: la barrera **impide** cruzar la frontera.
- μ grande: barrera "gorda", el mínimo queda lejos de la frontera (centro de la región). $\mu \to 0$: la barrera se pega a la frontera y $x_\mu \to x^*$.
- Curva $\{x_\mu\}$: **camino central** (central path), base de los **métodos de punto interior**.

Requisitos
Necesita un punto inicial **estrictamente factible** ($g < 0$) y un line search que **rechace** pasos que salgan de la región (en el código, $B = +\infty$ fuera).

Barrera y KKT perturbadas

En el mínimo x_μ: $\nabla f(x_\mu) + \sum_j \dfrac{\mu}{-g_j(x_\mu)} \nabla g_j(x_\mu) = 0$. Definiendo

$$\hat{\mu}_j := \frac{\mu}{-g_j(x_\mu)} > 0 \quad \Longrightarrow \quad \begin{cases} \nabla f + \sum_j \hat{\mu}_j \nabla g_j = 0 & \text{(estacionariedad ✓)} \\ g_j(x_\mu) < 0,\ \hat{\mu}_j > 0 & \text{(factibilidad ✓)} \\ \boxed{\hat{\mu}_j\, g_j(x_\mu) = -\mu} & \text{(complementariedad perturbada)} \end{cases}$$

Mensaje

La barrera resuelve las KKT con la complementariedad $\mu_j g_j = 0$ *relajada* a $\mu_j g_j = -\mu$. Cuando $\mu \to 0$ se recupera la KKT exacta, y $\hat{\mu}_j \to \mu_j^*$.

μ	x_1	x_2	f	máx g_i	$\hat{\mu}_1$	$\hat{\mu}_2$	cond.
10^{0}	0.47863	0.91189	2.32232	-6.1e-01	1.465	1.641	1.5e+00
10^{-1}	0.91186	0.96549	1.18524	-1.2e-01	0.746	0.815	2.2e+00
10^{-2}	0.99015	0.99523	1.01981	-1.5e-02	0.674	0.684	3.0e+00
10^{-3}	0.99900	0.99950	1.00200	-1.5e-03	0.667	0.668	3.1e+00
10^{-4}	0.99990	0.99995	1.00020	-1.5e-04	0.667	0.667	3.1e+00
10^{-5}	0.99999	1.00000	1.00002	-1.5e-05	0.667	0.667	3.1e+00
10^{-6}	1.00000	1.00000	1.00000	-1.5e-06	0.667	0.667	3.1e+00

P2 (barrera): máx $g_i \approx -1{,}5\,\mu$ y $\hat{\mu} \to (\frac{2}{3}, \frac{2}{3})$ ✓.

Exterior vs. interior

	Exterior ($\rho \uparrow$)	**Interior** ($\mu \downarrow$)
Iterados	**infactibles** (fuera)	**factibles** (dentro)
Punto inicial	cualquiera	estrictamente factible
Igualdades	sí, naturalmente	no directamente
Si se detiene antes	solución infactible	solución **usable** (factible)
Multiplicadores	$\rho\, g^+$	$-\mu/g$
Condicionamiento	$\sim \rho$	$\sim 1/\mu$
Región factible	puede tener interior vacío	necesita interior no vacío

Las barreras con búsqueda de camino central son el origen de los **métodos de punto interior** modernos (LP, QP, NLP).

Código: el solver interno (Newton amortiguado)

```
def newton(F, gradF, hessF, x0, tol=1e-10, max_iter=100):
    x = np.array(x0, float);  fx = F(x)
    for k in range(max_iter):
        g = gradF(x)
        if np.linalg.norm(g, np.inf) < tol:  return x
        H = hessF(x);  wmin = np.linalg.eigvalsh(H).min()
        if wmin < 1e-10:  H += (1e-6 - wmin) * np.eye(len(x))   # H > 0
        p = -np.linalg.solve(H, g);  a = 1.0
        while True:                                              # Armijo
            fn = F(x + a * p)
            if np.isfinite(fn) and fn <= fx + 1e-4*a*(g @ p) + 1e-14*(1+abs(fx)): break
            a *= 0.5
        x, fx = x + a * p, fn
    return x
```

Newton con line search (Clase 4); solo cambia la función F que se le pasa.

Código: Q (exterior) y B (interior)

```
def penalizacion_exterior(prob, x0, rhos):
    x = np.array(x0, float)
    for rho in rhos:                                          # rho = 1, 10, ..., 1e6
        Q    = lambda x: f(x) + 0.5*rho*np.sum(np.maximum(0, g(x))**2)
        gQ   = lambda x: grad(x) + rho * Jg(x).T @ np.maximum(0, g(x))
        hQ   = lambda x: hess(x) + rho*(np.einsum("j,jab->ab", np.maximum(0,g(x)), hessg(x))
                                        + Jg(x)[g(x) > 0].T @ Jg(x)[g(x) > 0])
        x = newton(Q, gQ, hQ, x)                              # warm start: x anterior
    return x

def barrera_interior(prob, x0, mus):
    x = np.array(x0, float)                                   # x0 estrictamente factible
    for mu in mus:                                            # mu = 1, 0.1, ..., 1e-5
        B  = lambda x: f(x) - mu*np.sum(np.log(-g(x))) if np.all(g(x) < 0) else np.inf
        gB = lambda x: grad(x) + mu * Jg(x).T @ (1.0 / -g(x))
        hB = lambda x: hess(x) + mu*(np.einsum("j,jab->ab", 1/-g(x), hessg(x))
                                     + Jg(x).T @ np.diag(1/g(x)**2) @ Jg(x))
        x = newton(B, gB, hB, x)
    return x
```

(Versión resumida de `scripts/14_penalizacion.py`; las Hessianas son analíticas.)

Resultados en P3 y comparación con `scipy`

- **Exterior**: 7 subproblemas ($\rho = 1, \ldots, 10^6$), 21 iteraciones de Newton en total; error final 6×10^{-7}, violación 10^{-6} (**infactible** por construcción).
- **Barrera**: 6 subproblemas ($\mu = 1, \ldots, 10^{-5}$), 38 iteraciones; error 4×10^{-6}, siempre factible.
- Comparadas con SQP (6 iteraciones, error 10^{-16}) hacen **más trabajo** y logran **menos precisión**: pagan el mal condicionamiento y el lazo externo.

`scipy.optimize`
`trust-constr` usa una barrera (punto interior) para desigualdades: en P3 converge con 13 iteraciones y error 10^{-5} (misma precisión que nuestra barrera con $\mu_{\text{mín}} = 10^{-5}$). No hay un "método de penalización" directo en `scipy`: es una técnica de **construcción** que se arma sobre cualquier minimizador sin restricciones, como hicimos aquí con Newton.

Más allá: penalidad exacta y Lagrangiano aumentado

- **Penalización ℓ_1 exacta**: $f + \nu\big(\sum |h_i| + \sum g_j^+\big)$. Con $\nu > \|\lambda^*\|_\infty$ el mínimo **es** x^* con ν **finito**: sin mal condicionamiento. Costo: no es derivable (de ahí viene la función de mérito de SQP).
- **Lagrangiano aumentado** (método de multiplicadores):

$$L_A(x, \lambda; \rho) = f + \lambda^\top h + \tfrac{\rho}{2}\|h\|^2, \qquad \lambda \leftarrow \lambda + \rho\, h(x)$$

  Con ρ **moderado** y λ actualizado converge sin $\rho \to \infty$. Es la base de ALGENCAN, LANCELOT, y de ADMM.

Moraleja
El defecto de la penalización clásica ($\rho \to \infty$) se corrige **estimando bien los multiplicadores**: otra vez aparecen λ, μ como protagonistas.

Resumen de penalización y barrera

- Penalización: problema con restricciones $\to$ **sucesión** de problemas sin restricciones; cada uno se resuelve con Newton/BFGS (Clase 4), con warm start.
- **Exterior**: $Q = f + \frac{\rho}{2}(\sum h^2 + \sum g_+^2)$; iterados infactibles; $\hat{\mu} = \rho g^+$; error $\sim 1/\rho$.
- **Interior**: $B = f - \mu \sum \log(-g)$; iterados factibles; $\hat{\mu} = -\mu/g$; KKT con complementariedad **perturbada**; error $\sim \mu$.
- Precio común: **condicionamiento** $\sim \rho$ o $1/\mu$ (cuando hay menos restricciones activas que variables).
- Simples de programar, **reusan** cualquier solver sin restricciones; pero menos eficientes que SQP cuando importa la precisión.

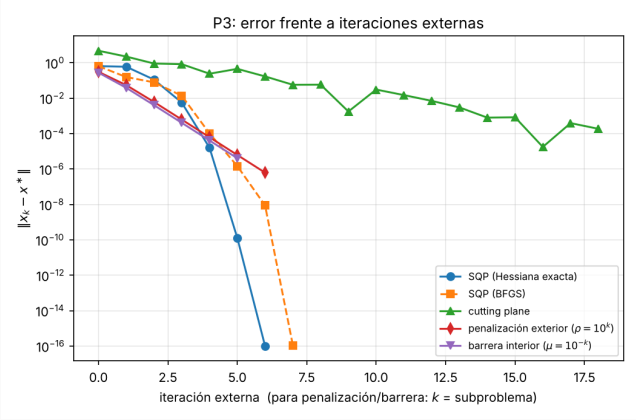
Síntesis y cierre del curso

Comparación final en P3 (convexo, óptimo en frontera curva)

Método	iter.	eval. f	eval. ∇f	$\|x - x^*\|$	viol.
SQP (Hessiana exacta)	6	19	13	0e+00	0e+00
SQP (BFGS)	7	30	29	1e-16	0e+00
Cutting plane (LP)	17	35	17	2e-05	7e-07
Penalización exterior	21	36	28	6e-07	1e-06
Barrera interior	38	52	44	4e-06	0e+00
scipy SLSQP (SQP)	9	11	9	3e-15	0e+00
scipy trust-constr (barrera)	13	8	8	1e-05	0e+00

- $x_0 = (0{,}3, 0{,}2)$ para todos. "iter." = iteraciones externas (QP, LP o pasos de Newton acumulados).
- Las evaluaciones son **aproximadas**: cada evaluación de la función de mérito cuenta como una de f.
- La métrica relevante es el **costo para una precisión dada**; no hay un "ganador" universal.

Convergencia comparada (P3)



- **SQP exacta**: caída superlineal/cuadrática, 6 pasos.
- **BFGS**: algo más lenta, sin segundas derivadas.
- **Cutting plane**: avance **lineal** y lento: cada corte solo mejora la aproximación poligonal.
- **Penalización/barrera**: error $\sim 1/\rho$ o $\sim \mu$: un decimal por subproblema.

P4: Rosenbrock en el disco (no convexo)

Método	iter.	eval. f	eval. ∇f	$\|x - x^*\|$	viol.
SQP (Hessiana exacta)	6	41	13	5e-09	0e+00
SQP (BFGS)	22	97	89	5e-09	0e+00
Penalización exterior	20	55	27	7e-08	1e-07
Barrera interior	52	65	58	4e-05	0e+00
scipy SLSQP (SQP)	17	23	17	5e-09	4e-13
scipy trust-constr (barrera)	36	27	25	2e-07	0e+00

- $x^* = (0{,}7864,\ 0{,}6177)$, $f^* = 0{,}0457$. **Cutting plane no aplica** (la función no es convexa: los cortes no subestiman).
- SQP exacta y `SLSQP` llegan a 5×10^{-9}; la barrera sufre cerca de la frontera con la curvatura de Rosenbrock (52 iteraciones).

Visión comparada de los métodos

Método	Subproblema	Convexidad	Convergencia	Iterados
Cutting plane	LP	necesaria	lenta	infactibles
SQP	QP	no necesaria	cuadr./superlineal	infactibles
Penal. exterior	sin restr.	no necesaria	lineal en ρ	infactibles
Barrera interior	sin restr.	no necesaria	lineal en μ	factibles

- **Todos** terminan en un punto que cumple las condiciones **KKT** (con las salvedades de convexidad).
- La diferencia está en **cómo** se aproxima el problema: lineal (CP), cuadrático (SQP), o eliminando restricciones (penalización).

Guía para elegir

- **Pocas variables, derivadas disponibles, alta precisión** $\rightarrow$ **SQP** (`SLSQP`).
- **Muchas variables y restricciones, estructura dispersa** $\rightarrow$ **punto interior** (`trust-constr`, IPOPT).
- **Problema convexo con restricciones complicadas, se quiere cota inferior** $\rightarrow$ **cutting plane** (pensar en planos de corte, Benders).
- **Solo se tiene un minimizador sin restricciones** $\rightarrow$ **penalización** (preferir ℓ_1 exacta o Lagrangiano aumentado).
- **Restricciones de caja** ($l \leq x \leq u$) $\rightarrow$ L-BFGS-B, `least_squares(bounds=...)`.

Siempre
Verificar KKT al final y mirar los **multiplicadores**: dicen qué restricciones importan y cuánto.

Cinco mensajes clave de la clase

1. **KKT**: en el óptimo $-\nabla f$ está en el cono de gradientes de restricciones **activas**; $\mu \geq 0$, $\mu_j g_j = 0$; los multiplicadores son **precios sombra**.
2. **QP**: con restricciones lineales y objetivo cuadrático, KKT es un sistema **lineal** (igualdades) o una búsqueda del conjunto activo (desigualdades).
3. **Cutting plane**: linearizar y resolver LPs; útil en convexos, lento por la curvatura.
4. **SQP** = Newton sobre KKT: un QP por iteración, con el **Lagrangiano** (no solo f) y función de mérito.
5. **Penalización/barrera**: eliminar restricciones a costa de **mal condicionamiento**; mejor con multiplicadores estimados (Lagrangiano aumentado).

Conexión con todo el curso

- **Sin restricciones** (Clases 1–4): gradiente, Newton, BFGS, CG, dogleg, LM.
- **Con restricciones** (Clase 5): se **reducen** a lo anterior: QP/LP por iteración (SQP, CP) o una sucesión de problemas libres (penalización).
- **Metaheurísticas** (Clase 3): no usan derivadas ni KKT; útiles cuando el problema es no diferenciable o discreto.
- Extensiones: gradiente conjugado con restricciones, optimización estocástica con restricciones, optimización robusta.

Ejercicios propuestos

1. Para mín $x_1^2 + x_2^2$ s.a. $x_1 + x_2 \geq 2$ plantee KKT y resuélvalo **a mano**. Interprete μ^* como precio sombra.
2. Resuelva un QP de 3 variables con 2 desigualdades usando el método de conjunto activo; compare con `scipy`.
3. Implemente cutting plane para mín $x_1 + x_2$ s.a. $x_1^2 + x_2^2 \leq 1$; grafique la cota inferior t_k vs. $f(x_k)$.
4. En SQP use $W = \nabla^2 f$ en vez de $\nabla^2_{xx} L$ en P3. ¿Qué ocurre? Explique.
5. Barrera: reproduzca la tabla de $\hat{\mu}$ para P3 y verifique $\hat{\mu}_j g_j = -\mu$.
6. Penalización exterior sobre P1: grafique la condición de la Hessiana vs. ρ y estime la pendiente en escala log-log.
7. (Retador) Implemente el **Lagrangiano aumentado** para P1 y compare con penalización cuadrática: ¿cuántos ρ necesita?

Referencias

- J. Nocedal y S. Wright, *Numerical Optimization*, 2.ª ed., Springer, 2006: cap. 12 (KKT), cap. 16 (QP), cap. 17 (penalización, barrera, Lagrangiano aumentado), cap. 18 (SQP).
- S. Boyd y L. Vandenberghe, *Convex Optimization*, Cambridge, 2004: cap. 5 (dualidad, KKT), cap. 11 (puntos interiores).
- D. Bertsekas, *Nonlinear Programming*, Athena Scientific, 1999.
- J. E. Kelley, "The cutting-plane method for solving convex programs", *J. SIAM* 8(4), 1960.
- D. Kraft, "A software package for sequential quadratic programming", DFVLR, 1988 (`SLSQP`).

- Código de la clase: `scripts/10_...15_*.py`; figuras en `figs/`.

¡Gracias!

Preguntas y comentarios